Chart.js

Objectifs pédagogiques

À l'issue de ce TD, vous serez capable de :

— intégrer la librairie Chart.js dans une page HTML ;
— créer et configurer différents types de graphiques ;
— personnaliser les options (axes, légende, styles) ;
— charger des données dynamiquement (API, AJAX, CSV) ;
— enrichir les graphiques avec des plugins et des indicateurs statistiques.

---

1 Installation et mise en place de Chart.js

1.1 Intégration de la librairie

Chart.js est généralement installé via `npm`. Pour ce TD, nous utiliserons une intégration via **CDN**.

1. Inclure Chart.js (version 4.x à défaut 3.x) dans le HTML à l'aide d'une balise `<script>`.
2. Ajouter un élément `<canvas>` avec un identifiant unique.

Attention : Les versions 2.x et antérieures de Chart.js utilisent une API différente (axes, options, plugins).

2 Premiers graphiques avec Chart.js

2.1 Découverte par l'exemple

Exécutez un premier exemple simple (diagramme circulaire) disponible en ligne.

2.2 Explorer d'autres types de graphiques

Travail demandé :

— Ajoutez deux nouveaux `canvas` dans votre HTML.
— Choisissez deux types de graphiques différents (bar, line, radar, etc.).
— Faites-les fonctionner dans votre page.

Les exemples fournis dans la documentation reposent sur :

— un objet `data` (données) ;
— un objet `config` (configuration).

Création du graphique :

```
new Chart('monCanvas', config);
```

2.3 Librairie Utils

Certains exemples utilisent une librairie utilitaire permettant de générer des données aléatoires.

— Télécharger `chart-utils.min.js` (sur e-campus).
— Lier le script avant ceux qui l'utilisent.
— Initialiser la librairie :

```
const Utils = ChartUtils.init();
```

Remarque : ne pas utiliser la partie *actions* (boutons) des exemples.

3 Configuration des graphiques

Un graphique Chart.js est défini par :

- `type` : type de graphique ;
- `data` : labels et jeux de données ;
- `plugins` : fonctionnalités supplémentaires ;
- `options` : personnalisation (axes, titre, légende).

Travail demandé :

- Modifier le titre d'un graphique.
- Configurer les axes.
- Tester différentes positions de légende.

Questions de synthèse

- Quelle est la différence entre `labels` et `datasets` ?
- À quoi sert `options.scales` ?

4 Exercice : visualisation des notes

Réalisez un graphique en barres représentant vos notes.
Contraintes :

- Axe des abscisses : matières ou semestres.
- Axe des ordonnées : notes de 0 à 20.
- 5 jeux de données correspondant à des compétences.
- 3 notes par compétence (S1, S2, S3).
- Couleurs différentes par compétence.
- Légende en haut du graphique.
- Padding de 20 pixels.
- Légende personnalisée (Arial, 14pt, rouge).

Questions de synthèse

- Comment fixer les bornes de l'axe Y ?
- Où se configure le style de la légende ?

5 Récupération de données distantes (API REST)

Nous utiliserons une API publique fournissant des produits fictifs.

`https://fakestoreapi.com/products/`

On peut filtrer les données en rajoutant des éléments dans l'URL, si l'API le permet (ici en précisant l'identifiant de l'employé par exemple). Le résultat est un fichier json. On peut le contacter depuis une page web via un appel ajax

```
fetch('https://fakestoreapi.com/products/')
.then(res => res.json())
.then(console.log);
```

ou avec jquery :

```
$.ajax({
      url:"https://fakestoreapi.com/products/",
   dataType: 'json',
   success:function(res){
    console.log(res);
  }
});
```

Travail demandé :

- Récupérer les données via `fetch` ou `$.ajax()`.
- Créer deux graphiques : il faudra limiter le nombre de caractères des abscisses avec par exemple la fonction javascript .slice(). Le deuxième, plus dur, donne en abscisse des tranches de prix, en ordonnée le nombre d'item correspondant.
  - produits / prix ;
  - tranches de prix / nombre de produits.

Questions de synthèse

- Qu'est-ce qu'une API REST ?
- Comment se fait le filtrage des données via une API REST ?

6 Mise à jour dynamique avec AJAX

Objectif : mettre à jour un graphique en fonction d'un formulaire.
Travail demandé :

- Créer une base `SAE303` contenant des notes (avec au moins 4 champs : un identifiant, la note elle-même, l'UE à laquelle elle correspond, à savoir la compétence, et le semestre auquel elle a été obtenue).
- Ajouter une liste déroulante des semestres.
- Charger dynamiquement les données correspondantes via AJAX.
- Détruire l'ancien graphique avant mise à jour (on ne visualise donc que les notes du semestre demandé).

Quelques indications :

- PHP peut retourner des données au format JSON en utilisant la fonction `json_encode()`. L'appel ajax `$.ajax()` a alors un champ `dataType:'json'` indiquant la nature des données attendues depuis le serveur.
- Il faudra détruire l'ancienne charte avant de la redessiner dans le même canvas, cela se fait avec la méthode `.destroy()`.

```
chart.destroy();
```

7 Chargement de données CSV

Le format CSV est couramment utilisé pour l'échange de données.
Travail demandé :

- Intégrer un plugin Chart.js pour charger un CSV.
- Ajouter un nouveau canvas.
- Tester avec un fichier CSV distant.

Questions de vérification

- Qu'est-ce qu'un fichier CSV ?
- Pourquoi l'accès aux fichiers locaux peut poser problème ?

8 Plugins Chart.js

Chart.js peut être enrichi grâce à des **plugins**, qui ajoutent des fonctionnalités non incluses nativement (annotations, affichage de valeurs, zoom, export, etc.).

Un plugin s'intègre généralement :

— via une balise `<script>` (CDN) après Chart.js ;

— ou via npm (non utilisé ici) ;

— puis via le champ `plugins` ou `options.plugins` de la configuration.

8.1 Plugin `chartjs-plugin-annotation`

Fonctionnalité : Permet d'ajouter des annotations visuelles sur un graphique :

— lignes horizontales ou verticales ;

— zones colorées ;

— repères de seuil (ex. : moyenne, note minimale, objectif).

Cas d'usage typique : Afficher une ligne correspondant à la moyenne des notes ou à un seuil de réussite (ex. 10/20).

Exemple de configuration (simplifié) :

```
options: {
  plugins: {
    annotation: {
      annotations: {
        seuil: {
          type: 'line',
          yMin: 10,
          yMax: 10
        }
      }
    }
  }
}
```

8.2 Plugin `chartjs-plugin-datalabels`

Fonctionnalité : Affiche directement les valeurs numériques sur les barres, points ou secteurs du graphique.

Cas d'usage typique :

— graphiques avec peu de données ;

— présentations où la valeur exacte est importante ;

— suppression de la nécessité de survoler les données.

Exemple de configuration :

```
options: {
  plugins: {
    datalabels: {
      color: 'black',
      font: {
        size: 12
      }
    }
  }
}
```

8.3 Plugin `chartjs-plugin-zoom`

Fonctionnalité : Permet de zoomer et de se déplacer dans un graphique à l'aide :

— de la molette de la souris ;

— du glisser-déposer ;

— d'actions tactiles.

Cas d'usage typique :

— graphiques contenant beaucoup de points ;

— données temporelles (séries chronologiques) ;

— analyse exploratoire des données.

Exemple de configuration :

```
options: {
  plugins: {
    zoom: {
      zoom: {
        wheel: {
          enabled: true
        }
      },
      pan: {
        enabled: true
      }
    }
  }
}
```

8.4 Travail demandé

— Choisissez au moins **un plugin** parmi ceux présentés.

— Intégrez-le dans votre projet Chart.js.

— Illustrez son utilité sur un graphique existant.

— Rédigez quelques lignes expliquant :

  — le problème résolu par le plugin ;

  — la valeur ajoutée par rapport à Chart.js seul.

Questions de synthèse

— Quelle différence faites-vous entre une option native et un plugin ?

— Pourquoi Chart.js repose-t-il fortement sur un écosystème de plugins ?

9 Indicateurs statistiques

Ajoutez une analyse statistique sous le graphique.
Travail demandé :

— Ajouter une div `stat`.

— Intégrer la librairie math.js.

— Afficher moyenne, variance et médiane d'un jeu de notes.